

# Biological Sequences

## Multiple sequence alignment

Dario Pescini

Università degli Studi di Milano-Bicocca

Dipartimento di Statistica e Metodi Quantitativi

[dario.pescini@unimib.it](mailto:dario.pescini@unimib.it)

# What is a multiple sequence alignment

## Multiple Sequence Alignment (MSA)

An **MSA** is a set of three or more biological sequences arranged in columns, such that each column contains *homologous* (evolutionarily related) residues.

- **Gaps** (-) are inserted wherever necessary to maintain column homology
- Main applications:
  - Phylogenetic analysis
  - Identification of conserved residues (e.g. active site)
  - Protein structure prediction (homology modelling)
  - Discovery of motifs and functional domains

# Three steps for an MSA with Biopython

- 1 **Step 1** — Prepare the input file in **multi-FASTA** format with all sequences to be aligned
- 2 **Step 2** — Run the external alignment tool (`clustalw2` or `muscle`) via the Biopython *wrapper* in `Bio.Align.Applications`
- 3 **Step 3** — Read the produced alignment with `Bio.AlignIO` and the guide tree with `Bio.Phylo`

## External dependencies

- `conda install -c bioconda clustalw`
- `conda install -c bioconda muscle`

# ClustalW — algorithm (overview)

ClustalW uses a **progressive** approach in three phases:

- 1 **Phase 1** — *Pairwise alignment*: aligns all possible pairs of sequences and computes a distance matrix  $D$
- 2 **Phase 2** — *Guide tree*: builds an approximate phylogenetic tree using the **Neighbor-Joining** method from  $D$
- 3 **Phase 3** — *Progressive alignment*: adds sequences to the MSA following the guide tree order; the most similar sequences are aligned first

The final result depends strongly on the guide tree: errors in early phases propagate (*frozen mistakes*).

# ClustalW — Neighbor Joining

## Neighbor Joining idea (Saitou & Nei, 1987)

A method for building an **unrooted tree** from a distance matrix:

- 1 Start from a *star tree* (all sequences connected to a central node)
- 2 At each iteration: find the pair  $(i, j)$  with the minimum net distance in matrix  $Q$
- 3 Join  $(i, j)$  into a new internal node  $u$ ; compute branch lengths  $d(i, u)$  and  $d(j, u)$
- 4 Update the distance matrix by removing  $i$  and  $j$  and adding  $u$
- 5 Repeat until only two nodes remain

The result is an unrooted tree; rooting requires an *outgroup*.

# ClustalW with Biopython

```
from Bio.Align.Applications import \
    ClustalwCommandline

clustalw_cline = ClustalwCommandline(
    'clustalw2',
    infile='data/multipleSequences.faa')

stdout, stderr = clustalw_cline()
```

Output files automatically produced in the same folder as the input:

- `multipleSequences.aln` — alignment in **Clustal** format
- `multipleSequences.dnd` — guide tree in **Newick** format

## Preparing the input file

The sequences to be aligned must be written into a single FASTA file:

```
from Bio import SeqIO
from Bio.SeqRecord import SeqRecord
from Bio.Seq import Seq

sequences = [
    SeqRecord(
        Seq('MSTQ...'),
        id='NP_014544',
        description='Saccharomyces cerevisiae'),
    # aggiungere le altre sequenze...
]
SeqIO.write(sequences,
            'data/multipleSeqs.faa', 'fasta')
```

- Alternatively: retrieve protein sequences of BLAST hits via `Entrez.efetch` and build `SeqRecord` objects from the result
- Extension `.faa` = *FASTA amino acid*

# MUSCLE — algorithm (overview)

MUSCLE (Edgar, 2004) uses a **three-stage** approach:

- 1 **Stage 1** — *Draft progressive*: compute  $k$ -mer distances between all pairs  
→ tree TREE1 → progressive alignment MSA1
- 2 **Stage 2** — *Improved progressive*: compute Kimura distances on MSA1  
→ tree TREE2 → re-align groups where the tree has changed
- 3 **Stage 3** — *Refinement*: iterative removal of an edge from the tree; the dataset is split into two subsets that are re-aligned; the new MSA is accepted only if the *SP score* improves

MUSCLE is generally more accurate and faster than ClustalW on large datasets.



# MUSCLE with Biopython

```
from Bio.Align.Applications import \
    MuscleCommandline

muscle_cline = MuscleCommandline(
    'muscle',
    input='data/multipleSeqs.faa',
    out='data/muscleAlignment.afa')

stdout, stderr = muscle_cline()
```

- Default output: **FASTA** format (extension .afa = *aligned FASTA*)
- For ClustalW format output: add `clw=True` and use extension .aln

## ClustalW output from MUSCLE

```
MuscleCommandline( ... , clw=True, out=' ... \.aln')
```

## Bio.AlignIO – reading the output

Bio.AlignIO is the equivalent of SeqIO for alignments:

```
from Bio import AlignIO

# Output da ClustalW (formato clustal)
align = AlignIO.read(
    'data/multipleSeqs.aln', 'clustal')
print(align)

# Output da MUSCLE (formato FASTA)
align = AlignIO.read(
    'data/muscleAlignment.afa', 'fasta')
print(align)
```

- AlignIO.read for a single alignment
- AlignIO.parse for files with multiple alignments
- Supported formats: 'clustal', 'fasta', 'phylip', 'stockholm', ...

## Exploring the alignment

```
print('Numero di sequenze:', len(alignment))
print('Lunghezza allineamento:',
      alignment.get_alignment_length())

for rec in alignment:
    print(rec.id, rec.seq[:30], '...')
```

- `len(alignment)` = number of sequences in the alignment
- `alignment.get_alignment_length()` = column length (including gaps)
- Each `rec` is a `SeqRecord` with `.id` and `.seq` (which includes – characters)
- `alignment[:,i]` = the  $i$ -th column of the alignment

## Reading a tree — Bio.Phylo

The guide tree produced by ClustalW is in **Newick** format:

```
from Bio import Phylo

# Albero Newick da ClustalW
tree = Phylo.read(
    'data/multipleSeqs.dnd', 'newick')

# Visualizzazione testuale ASCII
Phylo.draw_ascii(tree)
```

Formats supported by Bio.Phylo: 'newick', 'nexus', 'phyloxml',  
'nexml', ...

# Visualising the tree with matplotlib

```
tree.rooted = True  
Phylo.draw(tree)
```

- `tree.rooted = True` forces a rooted display
- `Phylo.draw(tree)` opens a matplotlib window
- To save to file, add:

```
import matplotlib.pyplot as plt  
plt.style.use('lecturesOliveFluo')  
fig = plt.gcf()  
fig.savefig('tree.pdf', bbox_inches='tight')
```

## Colouring branches

Convert to **PhyloXML** to support colours and widths:

```
tree = tree.as_phyloxml()  
tree.root.color = 'gray'  
  
# Trovare il MRCA di due sequenze e  
# colorare il sottoalbero  
mrca = tree.common_ancestor(  
    {'name': 'NP_014544'},  
    {'name': 'XP_004178593'})  
mrca.color = 'salmon'  
  
Phylo.draw(tree)
```

- `tree.as_phyloxml()` converts the Newick object to PhyloXML preserving the structure
- `common_ancestor` finds the deepest node that includes both sequences (MRCA)
- The colour propagates to all descendants of the node

# Saving the tree in PhyloXML

```
Phylo.write(tree,
            'data/tree_clustalw.xml',
            'phyloxml')

# Convertire in Newick (senza colori)
Phylo.convert(
    'data/tree_clustalw.xml', 'phyloxml',
    'data/tree_clustalw.nwk', 'newick')
```

## PhyloXML

- Preserves branch colours and widths
- Compatible with FigTree, Archaeopteryx

## Newick

- Simpler and more widely used format
- Does not support graphical metadata

# Complete MSA workflow

1. Collect protein sequences of BLAST hits: `Entrez.efetch` → `SeqRecord`
2. Write a multi-FASTA file with `SeqIO.write`
3. Align with `ClustalwCommandline` and `MuscleCommandline`
4. Read the alignment with `AlignIO.read`; explore length and columns
5. Read the tree with `Phylo.read`; visualise with `Phylo.draw_ascii` and `Phylo.draw`
6. Colour the MRCA subtree with `tree.as_phyloxml()` and `common_ancestor`
7. Save in PhyloXML and convert to Newick with `Phylo.write` and `Phylo.convert`